PROJECT PROPOSAL: PY-PACXON ANALYTICS ENGINE

1. Project Overview

Py-PacXon Analytics Engine is a grid-based arcade game developed using Python.
The player controls a character that moves around a grid map and captures territory by creating trails.

When the player leaves safe territory, a trail is created. If the player reconnects the trail back to captured territory, the enclosed area becomes captured.

*The objective of the game is to capture **at least 80% of the map** while avoiding enemy ghosts that move around the map. If a ghost touches the player or the player's trail, the player loses a life.*

This project focuses on:

- *practicing **Object-Oriented Programming***
- *implementing **game algorithms***
- *collecting **gameplay statistics***
- *analyzing **player behavior and risk strategies***

*The statistical data collected from gameplay will later be analyzed using Python tools such as **Pandas and Matplotlib**.*

2. Project Review

*Py-PacXon aims to recreate the core gameplay of the classic **Pac-Xon territory capture game** while focusing on programming learning objectives.*

The main goals of this project are:

- *To practice **game development using Python***
- *To apply **Object-Oriented Programming concepts***
- *To implement **algorithms such as Flood-Fill and collision detection***
- *To collect and analyze **gameplay statistical data***

*Compared to the original Pac-Xon game, this project focuses more on **learning programming concepts and data analysis** rather than building a full commercial game.*

3. Programming Development

3.1 Game Concept

Py-PacXon is a territory-capture game played on a 2D grid map.

Game Mechanics

1. *The player starts on safe territory.*
2. *When the player moves into empty space, a trail is created.*
3. *The player must reconnect the trail back to safe territory.*
4. *The enclosed area becomes captured.*

The player must capture 80% of the map to win the level.

Loss Conditions

The player loses a life when:

- *A ghost touches the player*
- *A ghost collides with the player's active trail*

3.2 Object-Oriented Programming Implementation

The system will use a hierarchical class structure consisting of multiple classes.

1. GameObject (Abstract)

Base class for all entities in the game.

Attributes

- `x` – *horizontal position*
- `y` – *vertical position*
- `color` – *sprite color*
- `speed` – *movement speed*

Methods

- `update()` – *updates object state*
- `draw()` – *renders object on screen*

2. Player (Inheritance)

Handles player control and trail logic.

Attributes

- `lives` – *number of remaining lives*
- `direction` – *movement direction*
- `is_trailing` – *indicates if the player is currently creating a trail*
- `trail_positions` – *list of grid positions forming the trail*

Methods

- `handle_input()` – *processes keyboard input*
- `move()` – *updates player position*
- `start_trail()` – *begins trail creation*
- `close_trail()` – *completes trail and triggers area*
- `lose_life()` – *reduces life count*

3. Enemy (Abstract)

Base class for ghost AI.

Attributes

- `x`, `y` – *ghost position*
- `velocity` – *movement speed*
- `direction` – *movement direction*

Methods

- `move()` – *controls ghost movement*
- `detect_collision(player)` – *checks collision with player or trail*

4. Ghost_Chaser (Polymorphism)

A ghost that attempts to move toward the player.

Methods

- `move()` – *overrides Enemy movement to chase the player*

5. Ghost_Bouncer (Polymorphism)

A ghost that moves in straight lines and bounces off walls.

Methods

- `move()` *– overrides Enemy movement with bouncing behavior*

6. GridManager

Handles grid logic and territory capture.

Attributes

- `grid` *– 2D grid representing captured and empty spaces*
- `captured_area` *– percentage of captured territory*

Methods

- `update_grid()` *– updates grid states*
- `flood_fill()` *– calculates captured regions*
- `calculate_coverage()` *– computes percentage of captured area*

7. GameEngine

Controls the main game loop and state transitions.

Attributes

- `game_state` *– current game state (menu, play, game over)*
- `score` *– player score*
- `level` *– current level*

Methods

- `run()` – *main game loop*
- `handle_collisions()` – *manages collision events*
- `update_game_state()` – *controls transitions between states*

8. StatsLogger

Responsible for recording gameplay data.

Attributes

- `file_path` – *location of CSV file*
- `data_buffer` – *temporary storage for records*

Methods

- `record_event()` – *records gameplay metrics*
- `write_csv()` – *writes data to CSV file*
- `reset_session()` – *clears stored data*

3.3 Algorithms Involved

Several algorithms will be used in the game.

Flood-Fill Algorithm

Used to determine which regions become captured when the player completes a trail.

The algorithm checks neighboring grid cells (up, down, left, right) to determine enclosed areas.

AABB Collision Detection

Axis-Aligned Bounding Box collision detection is used to detect collisions between:

- *player and ghost*
- *ghost and player trail*

Linear Interpolation (LERP)

LERP will be used to smooth the movement of characters between grid positions.

4. Statistical Data (Prop Stats)

4.1 Data Features

Examples of data that will be collected include:

1. Capture Efficiency

Amount of map area captured during each successful trail closure.

2. Risk Duration

Time spent outside safe territory while creating a trail.

3. Ghost Proximity

Distance between the player and the nearest ghost when the trail is completed.

4. Survival Interval

Time between gameplay events such as trail completion or player death.

5. Input Density

Number of direction changes made during a trail attempt.

4.2 Data Recording Method

The gameplay statistics will be recorded using the StatsLogger class.

When events occur (such as trail completion or player death):

1. *Gameplay statistics are collected*
2. *The data is stored temporarily*
3. *The data is written into a CSV file*

Using CSV files allows the data to be easily analyzed using Python libraries such as Pandas.

4.3 Data Analysis Report

The collected data will be analyzed using several types of graphs.

Capture Efficiency

Graph: ***Scatter Plot***

Reason: Used to analyze the relationship between risk duration and captured territory size.

Risk Duration

Graph: ***Line Chart***

Reason: Shows how risk time changes throughout gameplay.

Player Death Events

Graph: ***Bar Chart***

Reason: Used to compare the number of deaths across different game levels.

Ghost Proximity

Graph: ***Histogram***

Reason: Shows the distribution of distances between the player and ghosts during gameplay.

Input Density

Graph: ***Bar Chart***

Reason: Used to analyze how often the player changes direction during trail creation.

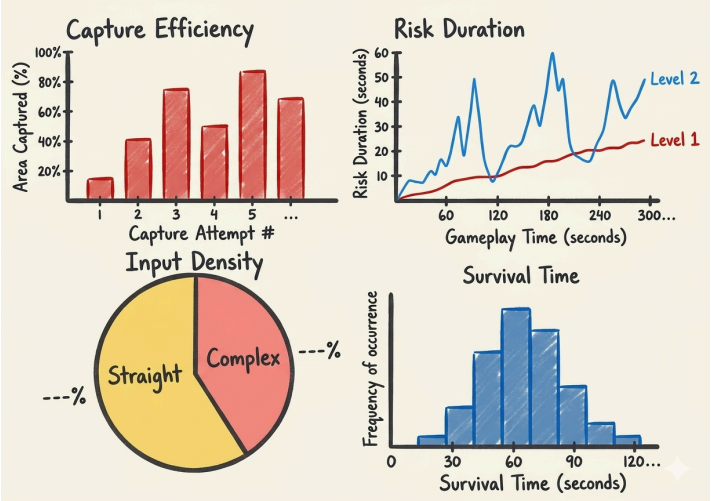
4.4 Update more about statistical Data

	Why It good to have this data	***How will obtain 100 value***	***Which variable or class I will collect***	***How will I display data***
Capture Efficiency	*To check the amount of grid area captured per trail; evaluates risk vs. reward.*	*Recorded every time a trail is successfully closed.*	*Collected in StatsLogger from GridManager.*	*Bar chart*
Risk Duration	*To measure time spent in the "danger zone" outside safe territory.*	*Recorded every time a player leaves a safe tile to start a trail.*	*Collected in StatsLogger from Player class.*	*Line chart*
Ghost Proximity	*To analyze how close the nearest ghost was when the trail closed.*	*Calculated at the exact moment of successful trail reconnection.*	*Collected in StatsLogger using player and ghost (x, y).*	*Scatter plot*
Input Density	*To check if the player uses straight lines or high-frequency directional changes.*	*Count every change in direction during an active trail attempt.*	*Collected in StatsLogger from handle_input in Player.*	*Pie chart*
Survival Time	*To measure consistency and performance duration across gameplay events.*	*Recorded as time between trail completions or player deaths.*	*Collected in StatsLogger from the GameEngine loop.*	*Histogram (Normal distribution)*

	Feature name	***Graph objective***	***Graph type***	***X - axis***	***Y - axis***
Graph 1	*Capture Efficiency*	*Show how much area the player captures per move.*	*Bar graph*	*Capture Attempt #*	*Area Captured (%)*
Graph 2	*Risk Duration*	*Compare time spent in danger zones across different levels.*	*Line graph*	*Gameplay Time (seconds)*	*Risk Duration (seconds)*
Graph 3	*Input Density*	*Show the proportion of movement styles (Straight vs. Complex).*	*Pie graph*	*None*	*None*
Graph 4	*Survival Time*	*Analyze the distribution and consistency of player lifespan.*	*Normal distribution*	*Survival Time (seconds)*	*Frequency of occurrence*

Gameplay Metric	***Count (n)***	***Mean (μ)***	***Std. Dev (σ)***	***Min***	***Max***
Capture Efficiency (%)	*100+*	*Average % per trail*	*Spread of capture size*	*Smallest trail*	*Largest trail*
Risk Duration (sec)	*100+*	*Avg time in danger*	*Consistency of risk*	*Quickest cap*	*Longest risk*
Ghost Proximity	*100+*	*Avg distance at close*	*Safety margin variance*	*Near miss*	*Max safety*
Survival Time (sec)	*100+*	*Average lifespan*	*Skill consistency*	*Early death*	*High score*

And this is my hand-drawn graph to see overall how it going to look like



5. Project Planning Timeline

Week	*Week*	*Task*
1	*8*	*Initial research and proposal draft.*
2	*9*	*Current: Final Proposal Submission & UML Design.*
3	*10*	*Logic Implementation:* `GridManager` *and Flood-Fill.*
4	*11*	*AI Development: Ghost movement and Collision Engine.*
5	*12*	*Stats Integration: Data collection and file I/O.*
6	*13*	*Testing Phase: Gameplay sessions to gather 100+ data rows.*
7	*14*	*Final Report & Video Presentation Submission.*

6. Document version

Version: 1.1
Editor: Pakhin Daonan 6810545859
Status: Pending Review